INDIAN INSTITUTE OF TECHNOLOGY ROORKEE

Distributed Systems

Design Issues
Issue 1: Time in Distributed Systems

Distributed system

- **Advantages**
  - Resource Sharing
  - Increased Performance/cost ratio
  - Fault Tolerance / Improved availability
  - Scalability
- **Inherent limitations of distributed systems**
  - ❖ Lack of common memory
  - ❖ Absence of a global (system wide) clock
  - ❖ Unpredictable communication delays

IIT ROORKEE

Examples of distributed system

- Client-Server or P2P?

	Client – Server
• Web Search	Y
• Stock Exchange	Y
• Massively Multiplayer online games / music / films	Y
• Travel Reservations	Y
• Skype	?
• WhatsApp messaging	?
• YouTube	Y
• Bitcoin	N

Distributed algorithm

- Algorithm on a single process (single thread of execution)
  - Sequence of steps taken to perform a computation.
  - *Steps are strictly sequential.*
- Distributed algorithm
  - Steps taken by each of the processes in the system (including transmission of messages).
  - *Different processes may execute their steps concurrently.*

Distributed Algorithms

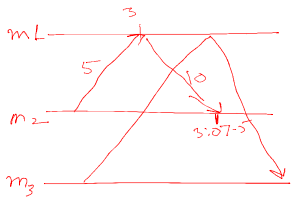
- Fundamental issues in the design of distributed algorithms are the following three factors.
  - **Asynchrony**: absolute and even relative timing of the events cannot be known precisely [2]
  - **Local view**: the computing entities can only be aware of the information it acquires, so it has only the local view of a global situation.
  - **Failures**: computing entities can fail independently, leaving some components operational while others are not
- These three factors complicate the design of the distribute algorithm

IIT ROORKEE

5

Notion of Global Time

- Absence of Global Clock (Time)
  - Assume a common clock
  - Synchronize physical clocks



NTP

IIT ROORKEE

6

Ordering of Events

Lamport's *Happened Before* relationship:

For two events a and b, a → b if

- a and b are events in the same process and a occurred before b
- a is a send event of a message m and b is the corresponding receive event at the destination process
- a → c and c → b for some event c

a → b implies a is a *potential* cause of b

Causal ordering : *potential* dependencies

"Happened Before" relationship causally orders events

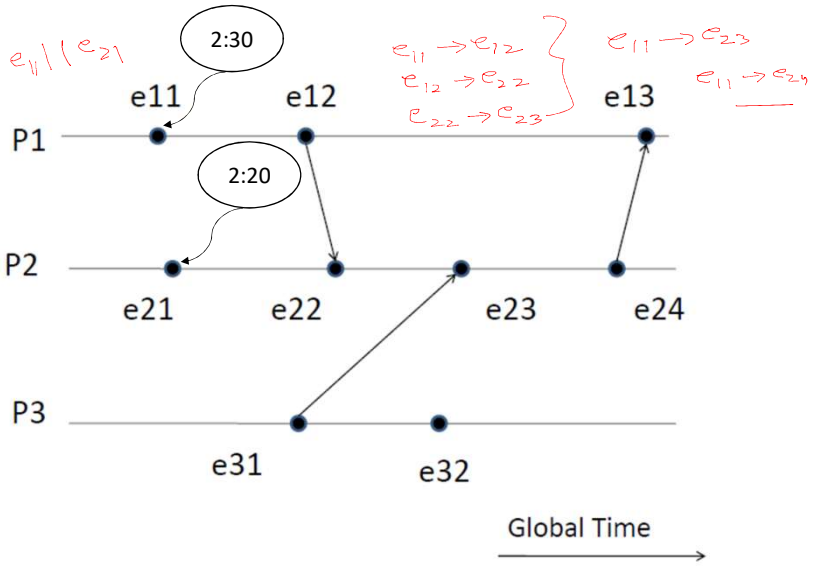
- If a → b, then a causally affects b

Concurrent events (a || b)

- If a ↛ b and b ↛ a, then a and b are concurrent

For any two events in a system

- Either a → b, b → a or a || b

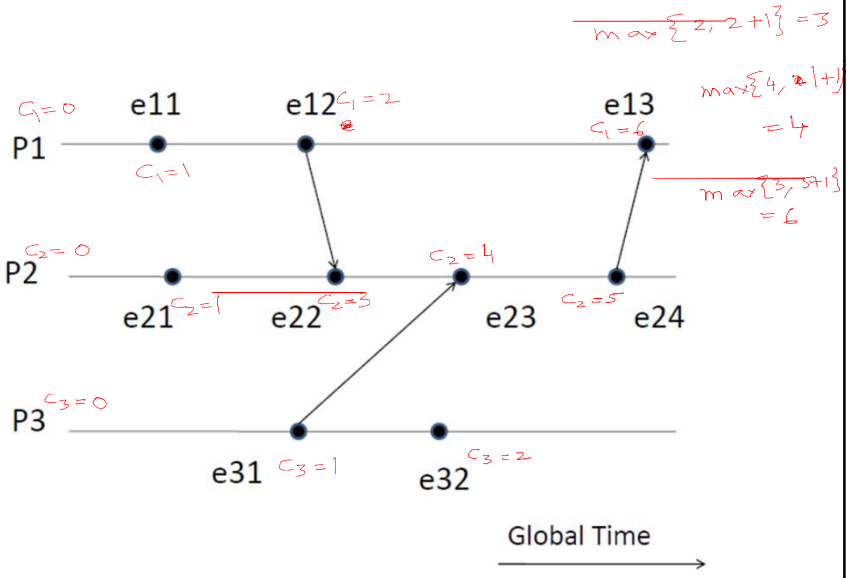


Lamport's Logical Clocks

Each process *i* keeps a clock C_i.

- Each event **a** in *i* is timestamped $C_i(\mathbf{a})$, the value of C_i when **a** occurred
- C_i is incremented by d (d > 0) for each event in *i*
- In addition, if **a** is a send of message m from process *i* to j, then on receive of m,

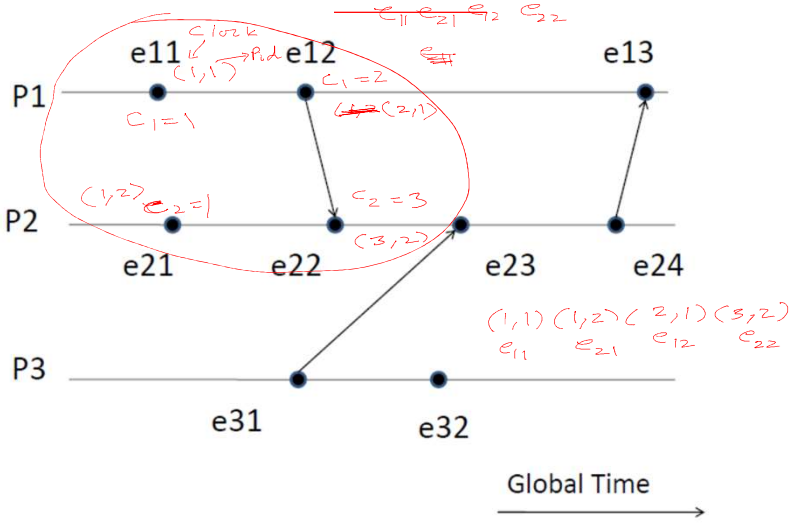
$C_j = \max(C_j, C_i(a)+d)$ //updated value of Cj



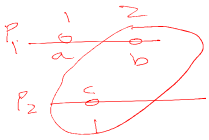
Conditions satisfied by the system of clocks

- if a → b, then C(a) < C(b)
  - For any two events a and b in a process *i*, if a occurs before b then $C_i(a) < C_i(b)$
  - If a is sending of a message m in *i* and b is receiving of m at process j, then $C_i(a) < C_j(b)$
- → is an irreflexive partial order
- A **reflexive**, **weak**, or **non-strict partial order** is a homogeneous relation ≤ on a set {P} that is reflexive, antisymmetric, and transitive.
- An **irreflexive**, **strong**, or **strict partial order** is a homogeneous relation < on a set {P} that is irreflexive, asymmetric, and transitive

- Total ordering (=>) possible by arbitrarily ordering concurrent events by process numbers



Limitation of Lamport's Clock



a → b implies C(a) < C(b)

BUT

C(a) < C(b) doesn't imply a → b !!

So not a true clock !!